Framework for Externalised Graph-Based Reasoning for Document-Level Question Answering

Akshat Kotadia
IIIT Hyderabad
2025201005

Gaurav Patel
IIIT Hyderabad
2025201065

Om Mehra
IIIT Hyderabad
2025201008

Hardik Kothari
IIIT Hyderabad
2025201046

Project Repository: https://github.com/Akshat-A-K/Graph-based-QA/

Abstract

Question answering over long documents needs both retrieval and reasoning over many pieces of text. We present a graph-based document QA system that first extracts PDF text, then creates a sentence graph, a span graph and a knowledge graph. The code uses pretrained models only. There is no training stage in the repository. The interactive interface is only a front end on top of the same core method. The system ranks evidence using BM25, dense embeddings, graph centrality, graph traversal, query expansion and knowledge graph matches. A pretrained extractive QA model gives the final answer span. On a balanced HotpotQA run of 900 questions, the system reaches 41.11% Exact Match and 48.24% F1. Bridge questions do better than comparison questions. This paper explains the method, the evaluation setup, the results and the main weak points of the system.

1 Introduction

1.1 Project Idea

The main idea of this work is to make document question answering more traceable. Instead of reading a long PDF as plain text only, the system first breaks it into sentences and smaller spans and then connects them with graph edges. This helps the model use document structure, semantic similarity and entity links together.

The system is evaluated in two modes only. The **Streamlit mode** is for interactive use and the **HotpotQA mode** is for benchmark evaluation. Both modes share the same graph-based method.

1.2 Contribution

The method does four things in a simple and practical way.

- It parses PDF files with text cleanup, optional OCR and optional table extraction.
- It builds sentence level, span level and knowledge graph level representations.
- It uses hybrid retrieval with BM25, dense embeddings, centrality, traversal, query expansion and KG evidence.
- It evaluates the system with both standard QA metrics and HotpotQA style analysis.

2 Related Work

The main benchmark for this work is **HotpotQA** (Yang et al., 2018). It was created for **multihop QA** and provides supporting facts. This is relevant to our use case as evidence reasoning is required, just like for our system, not simply the answer.

Tu et al. (2019) propose the heterogeneous document entity graph for multi-hop reading comprehension. They use different types of nodes and message-passing mechanisms similar to **GNNs** to collect evidence from multiple documents. The main similarity between our proposed model and theirs is that they both use the idea of using graphs for reasoning, whereas we don't train any graph neural network.

The **knowledge graph** survey by Kejriwal (2022) gives a practical view of the research landscape and explains how knowledge graphs support AI systems and many application areas. Peng et al. (2023) also give a useful survey and discuss the main challenges like embeddings, acquisition, completion, fusion and reasoning. These two papers are the most relevant KG surveys for this work because our system uses KG only as a lightweight reasoning layer over the document text.

For lexical retrieval, **BM25** is still a strong baseline (Robertson and Zaragoza, 2009). For

dense retrieval and text similarity, **SentenceBERT** (Reimers and Gurevych, 2019) is a common first step. **Dense Passage Retrieval** (Karpukhin et al., 2020) demonstrated the great potential of semantic retrieval for open domain QA while **REALM** (Guu et al., 2020) and **RAG** (Lewis et al., 2021) showed how to incorporate retrieval into language models for knowledge intensive tasks. **BERT style readers** (Devlin et al., 2019) remain the typical way to extract the final answer span.

3 Method

3.1 Two Execution Modes

The system is used in two modes only. The first mode is the interactive **Streamlit interface**, which is useful for manual inspection of one document and one question at a time. The second mode is the **HotpotQA** benchmark runner, which evaluates the same method on a fixed sample of questions. The core reasoning pipeline stays the same in both modes.

3.2 Method Overview

The system is a **graph-augmented question answering method** for long documents. It does not train a new model. Instead, it combines **pretrained language models** with **fixed graph construction rules** and **fixed scoring weights**.

The central idea is simple. A PDF is first converted into cleaned text. The text is then split into sentences and smaller spans. Those pieces are linked through semantic, structural and entity based edges. A retrieval layer then ranks the most useful evidence and a pretrained reader extracts the final answer span.

Figure 1 gives the end-to-end system view. It shows how the PDF parser, question processor and span extractor feed the document reasoning graph and knowledge graph before the reasoning engine produces the final answer. Table 1 summarizes the same pipeline at a more compact level.

Stage	Main idea
Cleaning text	Denoising before graph building including broken words and spaces artifacts.
Sentence and span split	Keep both coarse and fine evidence pieces for later ranking.
Construction of Graph	Link related text with semantic, structural and entity based edge.
Retrieval and Reading	Rank evidence, select the best contexts and extract the answer span.

Table 1: Conceptual view of the method.

3.3 Pretrained Models

The method uses standard pretrained models for representation, retrieval and reading. Table 2 lists the main ones used in the current code.

Component	**Model**
Embedding model	`BAAI/bge-large-en-v1.5` (Xiao et al., 2023)
Span embedding default	`multi-qa-mpnet-base-dot-v1` (Reimers and Gurevych, 2019) (Song et al., 2020)
NER model	`dslim/bert-base-NER` (Devlin et al., 2019)
KG spaCy model	`en_core_web_trf` (Honnibal and Montani, 2017)
QA reader	`deepset/deberta-v3-large-squad2` (He et al., 2021)
Cross encoder	`cross-encoder/ms-marco-MiniLM-L-6-v2` (Wang et al., 2020) (Bajaj et al., 2018)
Question classifier	`facebook/bart-large-mnli` (Lewis et al., 2019)
Question generator	`google/flan-t5-base` (Chung et al., 2022)

Table 2: Main pretrained models used in the method.

3.4 Document Representation

The first step is document cleaning and segmentation. The parser removes broken characters, hyphenation artefacts and extra spaces. It also keeps optional OCR and optional table extraction in mind, because some PDFs have useful answers only in scanned pages or tables or images. After cleaning, the text is split into sentences and then into smaller spans. This gives the method both coarse evidence and fine evidence.

The document graph is built at sentence level. Semantic similarity connects related sentences even when they do not appear next to each other. Structural links preserve order, page locality and section locality. Named entities add another signal, because they often connect the exact facts needed for a question. The graph uses a **semantic threshold of 0.75** and an adaptive cutoff based on the **85th%ile** of pairwise cosine scores. **Entity coreference edges use weight 0.85** and **KG overlap edges use weight 0.9**.

Span extraction adds more detail on top of sentence nodes. Full sentences are always kept and smaller spans are created from named entities, noun phrases, verb clauses and long clauses. This

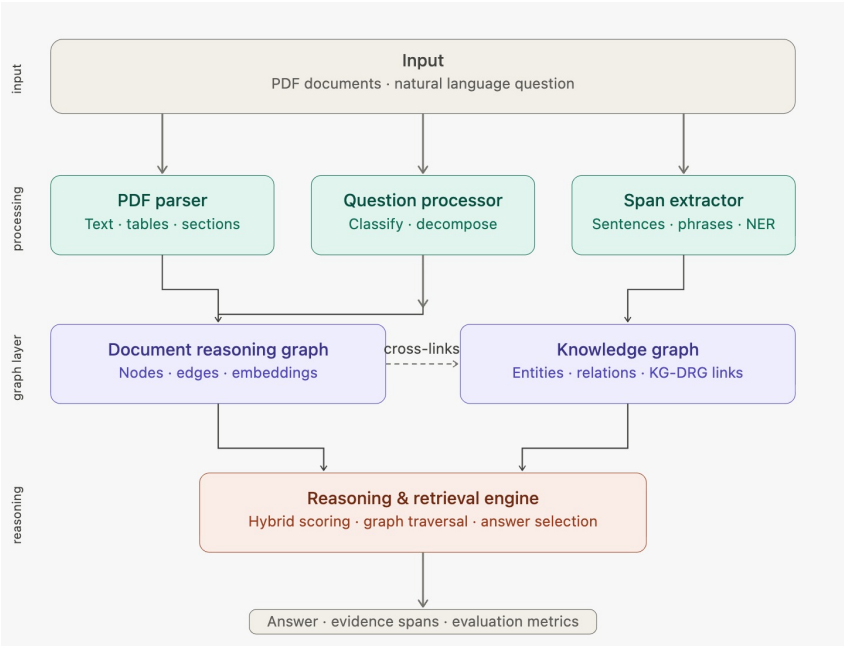

Figure 1: System architecture of the graph-based QA pipeline. The input PDF and question are routed through parsing, question processing and span extraction, then combined through the reasoning graph, knowledge graph and answer selection stage.

helps when the answer is a short entity inside a larger sentence. The span graph uses a **0.70** semantic threshold in the scripts, a **0.75** default in the class and an **80th%ile** adaptive cutoff.

3.5 Hybrid Reasoning and Answer Selection

Figure 2 highlights the core focus of the system: hybrid reasoning that fuses semantic, lexical and graph-based signals before reranking and answer selection.

The retrieval stage combines lexical search, dense similarity, graph traversal, query expansion and knowledge graph matching. BM25 gives the lexical signal. Dense embeddings give the semantic signal. Graph traversal adds nearby evidence from the document graph. Query expansion adds the top semantic spans. The knowledge graph helps when the question mentions an entity that appears in more than one sentence.

BM25 uses the standard setup with $k_1 = 1.5$ and $b = 0.75$. The query expansion step keeps the **top 3 semantic spans**. **Traversal depth** is **2** and the number of **neighbors per hop** is **5**.

The method ranks candidate spans with fixed weights. The sentence score uses **0.45 semantic**, **0.25 overlap**, **0.20 BM25** and **0.10 centrality**. The final candidate score uses **0.28 semantic**, **0.18 BM25**, **0.12 overlap**, **0.10 sentence bonus**, **0.08 centrality**, **0.06 hybrid**, **0.04 traversal**, **0.03 expansion**, **0.05 KG** and **0.06 KG bonus**. The **KG bonus** is capped at **0.15**.

Setting	Value
DRG semantic threshold	0.75 with 85th%ile adaptive cutoff
Span semantic threshold	0.70 in scripts, 0.75 class default
Span adaptive cutoff	80th%ile
BM25	$k_1 = 1.5$, $b = 0.75$
Traversal depth	2
Neighbors per hop	5
Query expansion	top 3 semantic spans
MMR lambda	0.65
QA max answer length	200
QA top k	3

Table 3: Main fixed settings used by the method.

To avoid returning near-duplicate evidence, the method also uses **Maximal Marginal Relevance**. The code uses $\lambda = 0.65$.

The final answer comes from a **pretrained extractive QA reader**. The reader uses `deepset/deberta-v3-large-squad2`, a **maximum answer length of 200** and **top** $k = 3$ candidate contexts.

3.6 Fixed Settings

The remaining important settings are listed in Table 3. They are fixed during evaluation and are not tuned on the benchmark.

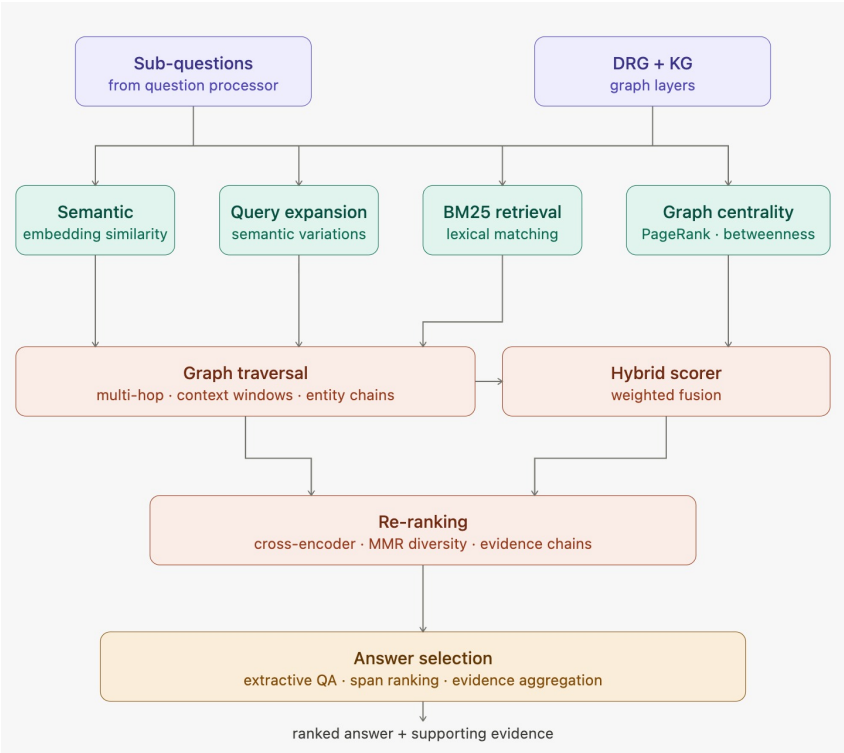

Figure 2: Hybrid reasoning pipeline used for retrieval and ranking. Semantic similarity, query expansion, BM25 and graph centrality feed graph traversal, a hybrid scorer and reranking before final answer selection.

Layer	Main role	Why it helps
Sentence graph	Keeps order and document flow	Good for nearby context and local evidence.
Span graph	Keeps short answer candidates	Useful when the answer sits inside a longer sentence.
Knowledge graph	Links repeated entities across the document	Helps multi-hop questions and entity heavy questions.

Table 4: How the three graph views contribute to the final method.

3.7 Design Rationale

This method uses **three graph views** because in long documents all important information is not in one place. **Sentence nodes** help to understand order and flow of the document. **Span nodes** help to find small answers and related facts. The **knowledge graph** is useful when the same names or entities come again in different parts. These three are not the same; they help in different ways. In the end using all of them together gives better result because they support each other. Table 4 summarizes the role of each layer.

Fixed thresholds are also important here. If we connect everything then the graph becomes too noisy. If we make it very strict then we miss useful links. So these thresholds are like a balance between covering enough data and keeping it clean. The same thing applies to the score weights. **BM25** works well for keyword type questions. **Dense similarity** helps when words are different but meaning is the same. **Traversal** and **expansion** help when the answer is not in the first sentence. **KG matching** gives small help when the question has many entities.

One more reason for this design is **explainability**. Each step has clear work and the output goes to the next step. So **debugging** becomes easy.

4 Experimental Setup

We evaluate the system on **HotpotQA** (Yang et al., 2018). The dataset has Wikipedia based question answer pairs and it is designed for **multi-hop reasoning**. It has two main question types, **bridge** and **comparison**. This is useful for our system because the method is also graph based and evidence based.

The benchmark runner evaluates **900 questions**. The sample is balanced across difficulty levels and question types, with **300 easy**, **300 medium** and **300 hard questions**. Each level contains **150 bridge questions** and **150 comparison questions**.

Metric	Main source
Exact Match	metric code
Precision/Recall/F1	metric code
Substring Match	metric code
Evidence Recall@k	metric code
Reasoning Depth	metric code
Time per Question	saved benchmark output
Total Pipeline Time	saved benchmark output
Avg KG Nodes/Edges	saved benchmark output

Table 5: Metrics reported by the current code and benchmark output.

5 Evaluation

5.1 Metrics

Table 5 lists the metrics that are used in the current implementation.

5.2 Experiment Details

The latest benchmark run evaluates **900 questions** for our graph method and **900 questions** for the Llama3 via Ollama baseline. The run uses `BAAI/bge-large-en-v1.5` for embeddings and `en_core_web_trf` for KG parsing. The average run time is still high because the graphs are rebuilt for every question.

5.3 Runtime Discussion

The runtime is mainly caused by repeated preprocessing. For every question, the system rebuilds the cleaned text, recomputes embeddings, regenerates graph links and runs the reader on multiple candidate contexts. The average time per question is **40.37 s**, so the design is accurate enough to study, but not yet fast enough for large scale interactive use. The total pipeline time is **616.9 minutes (37013 s)**, so the end-to-end cost remains substantial.

The graph statistics also help explain the runtime. The average DRG has **43 nodes and 43 edges**. The average span graph has **265 nodes and 2332 edges**. The average KG has **73 nodes and 56 edges**. That is not a huge graph, but the cost is spread across several models. Sentence embeddings, entity parsing, graph traversal and QA inference each add overhead. The pipeline is still **zero shot**, so there is no training cost but the inference path is heavy.

Even though each component is manageable, the overall time increases because many different processes run one after another for every query. Since everything happens during inference without prior training, the system ends up doing more work at runtime, which makes it slower.

6 Results and Analysis

6.1 Quantitative Results

Table 6 gives the full-width comparison of our graph method and the Llama3 baseline.

The combined table shows the main tradeoff clearly. Our graph method answers all **900 questions** and reaches **41.11% EM** and **48.24% F1**. It is ahead of Naive RAG on exact match, but the LLM baseline still has the strongest overall answer quality.

6.2 Comparison with Llama3 Baseline

The LLM setting remains the strongest baseline in the combined table, with **58.3% EM** and **70.4% F1**. **Naive RAG** sits at **33.3% EM** and **50.3% F1**. Our graph-based method is ahead of Naive RAG on exact match, but the LLM baseline is still stronger on F1 and answer quality overall. Table 7 restores the easy, medium and hard breakdown for our graph method, and Table 8 restores the same breakdown for the LLM baseline.

6.3 Level-wise Trends

The level-wise numbers show the same pattern in a more detailed way. Easy questions have the highest F1 and EM, medium questions has almost similar result as easy question and hard questions are clearly weaker on both metrics. The differences are modest between easy and medium, which suggests that the model is sensitive to question form as much as to nominal difficulty. Hard questions are the most difficult because the evidence chain is longer and the answer often depends on multiple small steps of reasoning.

The bridge and comparison splits also matter. **Bridge questions** reach **48.33% EM** and **51.24% F1**, while **comparison questions** reach **35.33% EM** and **45.23% F1**. The graph helps in bridge cases because it can walk from one evidence node to another. Comparison questions are harder because the model must not only find two entities, but also compare them correctly. A small mistake in entity matching or polarity can change the final answer.

The average reasoning depth of **6.05** also fits this picture. The method does not usually stop at one sentence, but it also does not wander too far. Most successful cases use a short chain of supporting

Metric	Our Graph	LLM	Naive RAG
Overall			
Questions evaluated	900	900	900
Answered	900 (100.0%)	891 (99.0%)	900 (100.0%)
Exact Match (EM)	0.4111	0.5833	0.3333
F1 Score	0.4824	0.7039	0.5033
Precision / Recall	0.5067 / 0.5162	0.7198 / 0.7275	0.4916 / 0.5855
Substring / R@5	0.5022 / 0.5589	0.7456 / N/A	0.5756 / 0.6667
EM bridge / compare	0.4833 / 0.3533	0.6311 / 0.5356	0.3956 / 0.2711
F1 bridge / compare	0.5124 / 0.4523	0.7574 / 0.6503	0.5170 / 0.4896
Avg reasoning depth	6.05	N/A	N/A
Avg DRG nodes/edges	43 / 43	N/A	N/A
Avg Span nodes/edges	265 / 2332	N/A	N/A
Avg KG nodes/edges	73 / 56	N/A	N/A
Avg time / question	40.37s	18.46s	3.86s
Total pipeline time	616.9min (37013s)	277.0min (16619s)	362.6min (21756s)

Table 6: Full-width comparison of our graph method with the Llama3 baseline.

Metric	Easy	Medium	Hard
EM	0.4833	0.4600	0.2900
F1	0.5324	0.5244	0.3904
Bridge EM	0.5800	0.4667	0.2400
Bridge F1	0.6810	0.5048	0.3515
Comparison EM	0.2667	0.4533	0.3400
Comparison F1	0.3838	0.5439	0.4293

Table 7: Difficulty breakdown of our graph method on 900 questions.

Difficulty	Metric	Ollama	RAG
Easy	EM	0.5733	0.4233
	Precision	0.7114	0.5773
	Recall	0.7178	0.6235
	F1	0.6926	0.5751
	Substring	0.7633	0.6400
Medium	EM	0.6667	0.3633
	Precision	0.7657	0.5036
	Recall	0.7922	0.6332
	F1	0.7649	0.5310
	Substring	0.7800	0.5933
Hard	EM	0.5100	0.2133
	Precision	0.6822	0.3941
	Recall	0.6725	0.4998
	F1	0.6541	0.4038
	Substring	0.6933	0.4933

Table 8: Difficulty-wise breakdown of Llama3 via Ollama baseline and Naive RAG.

spans. That is a good sign, because it means the graph layer is actually contributing to the answer path instead of only returning a single sentence.

6.4 Qualitative Cases

A few sample cases from the output show the pattern.

- **Correct bridge case**: *When did the series premier in which Blake Lee plays Josh Novak?* The gold answer is *October 1, 2017* and the system returns *October 1, 2017*. The bridge path is short and direct.
- **Correct comparison case**: *Which film was released first, Blackbeard's Ghost orThe Bears and I?* The gold answer is *Blackbeard's Ghost* and the system returns *Blackbeard's Ghost*. The comparison decision is correct here.
- **Partial answer case**: *A Tubeteika is a Central Asian cap, today worn in which Central Asia country, is the world's largest landlocked country, and the ninth largest in the world?* The gold answer is *Republic of Kazakhstan* but the system returns *Kazakhstan*. The answer is mostly right but the full entity form is lost.
- **Wrong polarity case**: *Are both documentaries The Big One and the Ring of Fire documentaries about natural disasters?* The gold answer is *no* but the system returns *yes*. This shows that comparison and negation handling are still weak.

These cases show that the system is good when the answer is in one strong span. It is weaker when the question needs exact entity normalization or a strict yes or no decision.

6.5 Discussion

The current system has three clear strengths.

- It gives an **answer for every question** in the run.

Case	Gold	Predicted	Takeaway
Bridge case	October 1, 2017	October 1, 2017	The short bridge chain is resolved correctly.
Compare case	Blackbeard's Ghost	Blackbeard's Ghost	The comparison decision is correct in this example.
Partial entity	Republic of Kazakhstan	Kazakhstan	The answer is close, but entity normalization is incomplete.
Polarity case	no	yes	Negation handling still fails on some yes/no questions.

Table 9: A few examples from the saved HotpotQA output.

- It works better on bridge questions than on comparison questions.
- It gives a useful **evidence trail** through spans and KG paths.

It also has two main weak points.

- The pipeline is **slow** because each question rebuilds the graphs.
- **Comparison and negation questions** still cause errors.

Overall, the results show that the graph based design is useful for document QA, but the system still depends a lot on preprocessing quality, entity normalization and the final reader.

6.6 Error Analysis

Table 9 shows a few example outputs from our model. The model performs well on bridge and comparison questions where reasoning is simple and direct. In partial entity cases, the model predicts incomplete answers (e.g., "Kazakhstan" instead of "Republic of Kazakhstan"), showing issues with exact matching. In polarity cases, it sometimes fails to handle negation and gives incorrect yes/no answers. Overall, the model works well for simple cases but needs improvement in entity precision and negation understanding.

6.7 Evidence Path Interpretation

The retrieved evidence usually falls into one of three patterns. In easy questions, one span is already enough to answer the question. In bridge questions, the first span often points to an entity and a second span completes the relation. In comparison questions, the retrieval step can find both entities, but the final reader still has to make a strict yes or no decision.

The average reasoning depth of **6.05** fits this behaviour. The system is not exploring a large search tree. That is a good balance for document QA, because very deep traversal can add noise. The graph layer seems to work best when it narrows the search space rather than trying to simulate full symbolic reasoning.

From the saved outputs, it's clear the system is already picking up the right kind of evidence most of the time. The small differences we see like slight wording changes or partially formed answers feel more like natural variation than actual mistakes.

7 Limitations

The main limitation is **runtime**. The system rebuilds the PDF text, the DRG, the span graph and the KG for every question. That design keeps the code simple, but it makes the average runtime high. The latest run takes **40.37 seconds per question**, so caching more intermediate structures would help a lot.

The second limitation is **answer normalization**. The reader can return a span that is semantically correct but not exactly the same as the gold answer string. This is visible in the entity form example where the system returns *Kazakhstan* instead of *Republic of Kazakhstan*. The current EM score penalizes this case even when the answer is nearly right.

The third limitation is **comparison reasoning**. Yes/no and same/different questions are harder than bridge questions in this code base. The retrieval step can still find relevant evidence, but the final decision can be wrong when the question contains negation, polarity, or a weakly stated comparison. A dedicated comparison head or a better evidence aggregation rule would help.

8 Conclusion

This paper studies a simple graph based QA system for long documents. The method uses document parsing, sentence graphs, span graphs, a knowledge graph, hybrid retrieval and an extractive reader. The final HotpotQA run gives **41.11% EM** and **48.24% F1** on **900 questions**. Bridge questions work better than comparison questions. The system is fully zero shot and does not need training, which makes it easy to run, but the tradeoff is lower accuracy on strict comparison and negation cases.

References

Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. 2018. Ms marco: A human generated machine reading comprehension dataset. *Preprint*, arXiv:1611.09268.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Sharan Narang, Gaurav Mishra, Adams Yu, and 12 others. 2022. Scaling instruction-finetuned language models. *arXiv preprint*.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. *Preprint*, arXiv:1810.04805.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. Realm: Retrieval-augmented language model pre-training. *Preprint*, arXiv:2002.08909.

Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2021. Deberta: Decoding-enhanced bert with disentangled attention. *Preprint*, arXiv:2006.03654.

Matthew Honnibal and Ines Montani. 2017. spacy 2: Natural language understanding with bloom embeddings, convolutional neural networks and incremental parsing.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. 2020. Dense passage retrieval for open-domain question answering. *Preprint*, arXiv:2004.04906.

Mayank Kejriwal. 2022. Knowledge graphs: A practical review of the research landscape. *Information*, 13(4):161.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *Preprint*, arXiv:1910.13461.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Preprint*, arXiv:2005.11401.

Ciyuan Peng, Feng Xia, Mehdi Naseriparsa, and Francesco Osborne. 2023. Knowledge graphs: Opportunities and challenges. *Artificial Intelligence Review*, 56(11):13071–13102.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *Preprint*, arXiv:1908.10084.

Stephen E. Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: Bm25 and beyond. *Found. Trends Inf. Retr.*, 3:333–389.

Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2020. Mpnet: Masked and permuted pre-training for language understanding. *Preprint*, arXiv:2004.09297.

Ming Tu, Guangtao Wang, Jing Huang, Yun Tang, Xiaodong He, and Bowen Zhou. 2019. Multi-hop reading comprehension across multiple documents by reasoning over heterogeneous graphs. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2704–2713, Florence, Italy. Association for Computational Linguistics.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Preprint*, arXiv:2002.10957.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-pack: Packaged resources to advance general chinese embedding. *Preprint*, arXiv:2309.07597.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium. Association for Computational Linguistics.